

MATH 3190 Homework 2

Due February 5, 2026

Focus: Notes 4

Total Points: 65 + 10 (Self-Critique Comment and Uploading to GitHub) = 75

Note that your homework should be completed in Quarto. You can just add your answers and code to this document in the appropriate part. The file should be rendered as an html document or pdf document (html is quicker to render and you will need an installation of LaTeX to render as a pdf). You will “turn in” this homework by uploading the files, the .qmd and output file, to your GitHub `Math_3190_Assignment` repository in the `Homework/Homework_2` directory. Make sure all supporting files (if there are any) are in this directory as well. You should use `git` and `Unix` commands in a terminal for this (i.e. don’t manually upload files to GitHub). In addition, you will submit just your html or pdf file on Canvas to unlock the solutions.

Don’t forget to also put a self-critique comment on Canvas indicating what you did correctly and incorrectly and include any questions you may have about the assignment after submitting the assignment and thoroughly looking through the solutions.

Problem 1 (65 points)

It is time to practice using SQL. First, we will need some files that are worthy of being treated as databases. For this, you will need a little more than 1 GB of hard drive space. Alternatively, you should be able to store these files on your OneDrive. Go ahead and download all of the files from this link: https://suu-my.sharepoint.com/:f/g/personal/richardbrown1_suu_edu/IgCX2W7pUzK1QIsVpB1dbIF_AQc_ii3Jwm9eHWDBm2xCAYk?e=GeszqA. While the `airports.csv` and `airlines.csv` files are quite small, the `flights.csv` is over 565 MB in size and contains the records of 5,819,079 flights in USA in 2015. These data came from the [Bureau of Transportation Statistics](#). **Make sure you save these file outside of your `Math_3190_Assignments` since you will not be able to push the `flights.csv` file to GitHub due to its size.**

Part a (5 points)

While it is probable that your computer could read in this data set in its entirety, we are going to avoid that and instead make a database that we can access using SQL. Follow the example on slides 55-57 of Notes 4 to create a `.sqlite` file that contains the tables `flights` (from `flights.csv`), `airports` (from `airports.csv`), and `airlines` (from `airlines.csv`). **Again, make sure you save this `.sqlite` file outside of your `Math_3190_Assignments` since you will not be able to push it to GitHub due to its size.**

Part b (2 points)

Connect to the `.sqlite` database that you created in Part a like we did on slide 52 of Notes 5. Then use the `dbListTable()` function on that connection to list all the tables in the database. There should be three tables that you added in Part a.

Part c (1 point)

There are many different types of SQL databases: MySQL, SQLite, SQL Server, etc. We created a SQLite database in Part a. A way to see all the columns of a table in an SQLite database is with the command `PRAGMA table_info('table_name')`. Using your connection from Part b, use the `dbGetQuery()` function in an **R** code chunk to print all the column names from the `flights` table. This is useful when we have a large file that is hard to even open to see what variables are in there.

Part d (24 points)

Again using the `dbGetQuery()` function (one query per part) in an **R** code chunk, use SQL statements to answer the following questions. Note that distances are in miles.

1. What is the average distance of all flights in the US in 2015?
2. What is the minimum flight distance of all flights in the US in 2015?
3. What is the maximum flight distance of all flights in the US in 2015?
4. Between what two airports did the minimum distance flight take place?
5. Between what two airports did the maximum distance flight take place? The `SELECT DISTINCT` command will be useful here.
6. You might notice that some airports are coded with numbers instead of their IATA codes. Repeat item 5, but do not include the airports that begin with a “1”.

7. While you may know what these airports codes are, it would be nice if the full name of the airport was printed. The airport code information is in the `airport` table in your database in the `IATA_CODE` variable. Use the `INNER JOIN` statement to output the airport name for the two airports that have the minimum flight distance. Note that this command can take a few seconds to run.
8. Repeat item 7 with the maximum flight distances.

Part e (11 points)

Many of these queries are generating outputs that are fairly small. That is, they are easily able to be stored as variables in **R**. A couple hundred thousand rows is fine to work with since that only takes a couple MB of RAM.

1. Using the `dbGetQuery()` function again, obtain the arrival delay times and the name of the airline (by joining with the `airlines` table) for all the data entries that have a delay time more than 60 minutes. Save this output as a tibble in **R**.
2. Once you have this, make side-by-side boxplots in `ggplot()` for the arrival delay time split by the airline. This plot won't look too great unless the y-axis is scaled. Go ahead and use the `log10` scale like we did back in Lab 2. Then look up how to angle the axis labels so they do not overlap and incorporate that in your plot.

Part f (10 points)

1. Now, let's NOT use the `dbGetQuery()` function. Instead, insert an SQL code chunk for this part of the problem that selects the **average** arrival delay (for some reason, there is not a median function built in to SQL) and the airline's full name for each airline. So, this output should only have 14 rows.
2. Then pass this output back to **R** and create a barplot with the airlines on the x-axis and the average arrival delay on the y-axis. Again, angle the x-axis labels so they do not overlap.

Part g (12 points)

1. Now let's use the `dbplyr` add-on to the `dplyr` package to work with an SQL database directly. You may have to install the `dbplyr` package. Use the `tbl()` function to create an object you can work with directly.

2. With that object, using **R** commands from the tidyverse, find the average and standard deviation of the elapsed time of the flights by airline and arrange it by the SD (with the largest at the top). Make sure to use the `inner_join()` function so that the name of the airline, not just its code, is given. Note that to save this object in **R**'s memory, you will need to do something like pipe it into the `collect()` function.
3. Finally, make a ggplot bar graph like the one you made in Part f, but use the standard deviation as the heights on the y-axis. Why do you suppose the airline with the largest standard deviation in elapsed time has such variable flight times?

Finally, when you are done with this assignment, it is good practice to close the connection to the flights database.